Azure Service Bus – Peek-Lock Mode vs Receive-and-Delete Mode

1. Overview

Azure Service Bus provides different **receive modes** for reading messages from a queue.

The two important receive modes are:

1. **Peek-Lock**
2. **Receive-and-Delete**

The main difference is **what happens to the message after it is received**.

Receive Mode	Message Deleted Immediately?	Message Completion Required?
Peek-Lock	No	Yes
Receive-and-Delete	Yes	No

---

2. Peek-Lock Mode

In **Peek-Lock mode**, when a message is received from the queue, the message is **not immediately deleted**.

Instead, Azure Service Bus places a lock on the message.

The application can process the message and then explicitly tell Service Bus that the message has been successfully processed.

Flow

```
Message in Queue
       ↓
Receive Message
       ↓
Message is Locked
       ↓
Process Message
       ↓
Complete Message
```

```
        ↓
Message is Removed from Queue
```

Important Point

Receiving a message does **not** mean that the message has been deleted.

We need to explicitly complete the message.

For example:

```
await receiver.CompleteMessageAsync(message);
```

Once `CompleteMessageAsync()` is called successfully, the message is removed from the queue.

---

3. Why is Peek-Lock Useful?

Peek-Lock is useful when message processing is important and we don't want to lose a message if processing fails.

Example

Suppose an order message is received:

```
OrderId: 1001
Amount: ₹5000
```

The application receives the message and starts processing the order.

If the application crashes while processing the message, the message has **not been completed**.

After the lock expires, the message can become available again for processing.

Therefore, Peek-Lock provides better reliability for important business operations.

---

4. Receive-and-Delete Mode

In **Receive-and-Delete mode**, the message is deleted from the queue as soon as it is successfully received.

Flow

```
Message in Queue
       ↓
Receive Message
       ↓
Message is Deleted
       ↓
Process Message
```

There is no separate `CompleteMessageAsync()` operation required.

Important Point

Once the message is received successfully, it is removed from the queue.

Therefore, if the application crashes after receiving the message but before processing it, the message may be lost.

---

5. Peek-Lock vs Receive-and-Delete

Peek-Lock

```
Queue
  ↓
Receive
  ↓
Lock Message
  ↓
Process
  ↓
Complete
  ↓
Delete
```

The message remains recoverable until it is successfully completed.

Advantages

- Safer for important messages
- Supports reliable message processing
- Message can be retried if processing fails

- Suitable for business-critical operations

Disadvantage

- Requires explicit message completion
- More processing logic is required

Receive-and-Delete

```
Queue
  ↓
Receive
  ↓
Delete
  ↓
Process
```

Advantages

- Simple
- No need to explicitly complete the message
- Suitable when message loss is acceptable

Disadvantage

- Message can be lost if processing fails after receiving it
- Not suitable for critical business operations where every message must be processed

---

6. Creating a Receiver in C

The receiver can be created using `ServiceBusClient`.

Peek-Lock Receiver

```
ServiceBusReceiver receiver = serviceBusClient.CreateReceiver(
    queueName,
    new ServiceBusReceiverOptions
    {
        ReceiveMode = ServiceBusReceiveMode.PeekLock
    });
```

Here:

```
ReceiveMode = ServiceBusReceiveMode.PeekLock
```

specifies that the receiver should use Peek-Lock mode.

Default Behavior

If no receive mode is specified, **Peek-Lock is the default receive mode**.

So this:

```
var receiver = serviceBusClient.CreateReceiver(queueName);
```

uses Peek-Lock by default.

---

7. Receiving a Message in Peek-Lock Mode

Example:

```
ServiceBusMessage message = await receiver.ReceiveMessageAsync();
```

The message is received and locked, but it is **not deleted yet**.

After successful processing:

```
await receiver.CompleteMessageAsync(message);
```

Now the message is removed from the queue.

---

8. Receive-and-Delete Receiver

To explicitly configure Receive-and-Delete mode:

```
ServiceBusReceiver receiver = serviceBusClient.CreateReceiver(
    queueName,
    new ServiceBusReceiverOptions
```

```
    {
        ReceiveMode = ServiceBusReceiveMode.ReceiveAndDelete
    });
```

Here:

```
ReceiveMode = ServiceBusReceiveMode.ReceiveAndDelete
```

means the message will be deleted when it is received.

---

9. Example Scenario

Suppose the queue contains:

```
Message 1
```

Using Peek-Lock

Application receives:

```
Message 1
```

The queue still contains the message while it is locked.

```
Queue
 └── Message 1 (Locked)
```

After successful processing:

```
await receiver.CompleteMessageAsync(message);
```

The message is removed:

```
Queue
 └── Empty
```

Using Receive-and-Delete

Application receives:

```
Message 1
```

Immediately after receiving:

```
Queue
 └── Empty
```

The message has already been deleted.

---

10. Practical Example from the Video

The queue initially contained **one message**.

Step 1 – Check the Queue

Azure Portal showed:

```
Queue: Q1
Messages: 1
```

Step 2 – Receive Using Peek-Lock

The application received the message using:

```
ServiceBusReceiveMode.PeekLock
```

The message body was successfully received.

However, after refreshing the Azure Portal, the message was still present.

```
Before receiving:
Messages = 1
```

```
After Peek-Lock receive:
Messages = 1
```

This proves that **Peek-Lock does not automatically delete the message**.

Step 3 – Receive Using Receive-and-Delete

The same message was then received using:

```
ServiceBusReceiveMode.ReceiveAndDelete
```

The message was successfully received.

After refreshing the Azure Portal:

```
Messages = 0
```

The message was deleted from the queue.

---

11. Easy Way to Remember

Peek-Lock

Receive → Lock → Process → Complete → Delete

Think:

> "I will keep the message safe until I finish processing it."

Receive-and-Delete

Receive → Delete → Process

Think:

> "Delete the message as soon as I receive it."

---

12. Interview Question

Q: What is the difference between Peek-Lock and Receive-and-Delete?

Answer:

In **Peek-Lock mode**, the message is locked when received but is not deleted immediately. After successfully processing the message, we explicitly complete it using `CompleteMessageAsync()`. If processing fails, the message can become available again after the lock expires.

In **Receive-and-Delete mode**, the message is deleted immediately after it is successfully received. Therefore, if processing fails after receiving the message, the message cannot be recovered from the queue.

---

13. Which One Should We Use?

For most **business-critical applications**, Peek-Lock is preferred because it provides safer message processing.

Use Peek-Lock when:

- Every message is important
- Processing can fail
- You need retry behavior
- You need reliable message processing
- You want to explicitly complete or abandon messages

Use Receive-and-Delete when:

- Losing a message is acceptable
- Processing is very simple
- You don't need retry behavior
- You want simpler message handling

---

14. Key Points to Remember

- Azure Service Bus supports **Peek-Lock** and **Receive-and-Delete** receive modes.
- **Peek-Lock is the default receive mode.**
- Peek-Lock does **not** immediately delete the message.
- The message is temporarily locked while being processed.
- After successful processing, call:

```
await receiver.CompleteMessageAsync(message);
```

- Receive-and-Delete deletes the message immediately after receiving it.
- Receive-and-Delete does not require `CompleteMessageAsync()`.
- Peek-Lock is generally safer for important business messages.
- Receive-and-Delete is suitable when message loss is acceptable.

Quick Memory Trick

```
PEEK-LOCK
Receive → Lock → Process → Complete → Delete

RECEIVE-AND-DELETE
Receive → Delete → Process
```